

user@linux:~\$

# Coding Logics

*A Linux Guide to Building Formal Systems in C*

C99 · gcc · Linux

James Alexander Pugmire · Propagation Logic Project · 2026

Standalone — everything you need is in this book — no other documents required

---

## PREFACE

### Why C on Linux

*The ideas are simple. The machine is real. Use both.*

---

Linux is built in C. The kernel, the shell, the standard library — all C. When you compile a program on Linux you are using tools that were written in the same language, running on an operating system written in the same language, on hardware whose firmware was probably written in the same language. There is a vertical coherence to this that most programming environments do not have.

Formal logic ought to have the same property. A logical system should be something you can build from scratch, inspect completely, and verify without trusting anyone else's framework. C on Linux gives you that. A carrier set — the object this book is about — fits in about two hundred lines of C. No dependencies. No package manager. No runtime. Just a source file, a compiler, and a terminal.

This book is standalone. You do not need to have read anything else. Chapter 1 explains what a carrier set is from the ground up. Chapter 0 gets you set up if you have not compiled C before. Appendix C gives the mathematical reference if you want the formal definitions. Everything else is code that compiles and runs.

> *C code written to the C99 standard compiles on any Linux system made in the last twenty-five years.  
The code in this book will still work in 2050.*

**Who this book is for.** Anyone using Linux who is curious about formal logic, wants to understand it mechanically rather than philosophically, and is comfortable with a terminal. You need to know: what a variable is, what a function is, what a loop is. You do not need prior C experience — the language constructs used here are introduced as they appear.

**Companion books.** *Coding Logics in Python* covers the same ten ideas in Python. *Coding Logics in Rust* shows how a static type system enforces carrier properties the C version only checks at runtime. Neither is required to follow this book.

## Setup: Terminal, GCC, First Compile

*Everything you need installed before Chapter 1.*

---

### Opening a terminal

---

On most Linux desktops: **Ctrl+Alt+T**. On a server or WSL: you are already there. The dollar sign `$` is the prompt. Everything after it is a command you type.

### Installing GCC

---

GCC is the GNU C Compiler. Check whether it is installed:

```
$ gcc --version
```

If you see a version number, you are ready. If you see *command not found*, install it:

```
# Debian, Ubuntu, Linux Mint, Pop!_OS: sudo apt install gcc # Fedora, RHEL, CentOS  
Stream: sudo dnf install gcc # Arch, Manjaro, EndeavourOS: sudo pacman -S gcc #  
openSUSE: sudo zypper install gcc
```

### Your first C program

---

Create a file. You can use any editor — `nano`, `vim`, `gedit`, VS Code with the terminal open alongside it. The choice does not matter for this book.

```
$ nano hello.c
```

`hello.c`

```
#include <stdio.h>

int main(void) {
    printf("hello, logic\n");
    return 0;
}
```

Compile it:

```
$ gcc -Wall -o hello hello.c
```

Run it:

```
$ ./hello
```

```
OUTPUT
hello, logic
```

**What those flags mean.** `-Wall` enables all warnings. `-o hello` names the output file `hello`. Always compile with `-Wall`. Warnings are the compiler telling you something is likely wrong. Treat them as errors until you understand what they mean.

### The flags we use throughout this book

---

```
$ gcc -Wall -Wextra -std=c99 -lm -o carrier carrier.c
```

`-Wextra`: extra warnings beyond `-Wall`.

`-std=c99`: use the C99 standard (adds `bool`, better for-loops, and more).

`-lm`: link the maths library (needed for `fabs()` in `math.h`).

### A minimal Makefile

---

A Makefile lets you type `make` instead of the full compiler command every time. Create it in the same directory:

#### Makefile

```
CC      = gcc
CFLAGS  = -Wall -Wextra -std=c99 -lm

carrier: carrier.c
    $(CC) $(CFLAGS) -o carrier carrier.c

clean:
    rm -f carrier

.PHONY: clean
```

The indentation before `$(CC)` must be a **tab character**, not spaces. Make is sensitive to this. Most editors insert tabs correctly in Makefiles, but if you see errors, check.

Now `make` compiles and `make clean` removes the binary.

#### EXERCISE 0

Compile and run `hello.c`. Then deliberately introduce a mistake — remove the closing brace of `main` — and recompile. Read the error message. Then fix it. Getting comfortable with compiler error messages is the first skill.

## What Is a Carrier?

*The idea before the code. Read this once, carefully.*

---

A *carrier set* is a value space plus three operations plus a threshold. That is all. Everything in formal logic — every theorem, every paradox, every system of rules — is an output of those five things.

Here is the classical carrier you already know:

- **Values (V):**  $\{0, 1\}$  — false and true
- **NOT:** flip the value.  $\text{NOT}(0) = 1$ ,  $\text{NOT}(1) = 0$
- **AND:** multiply.  $1 \text{ AND } 1 = 1$ , anything else  $= 0$
- **OR:** maximum.  $\max(1, 0) = 1$ ,  $\max(0, 0) = 0$
- **Threshold ( $\theta$ ):** 1.0 — how much load the system tolerates

From these five choices, the following become unavoidable facts of arithmetic:

- **Law of Non-Contradiction:**  $v \text{ AND NOT}(v) = 0$  for all  $v$  in  $V$
- **Law of Excluded Middle:**  $v \text{ OR NOT}(v) = 1$  for all  $v$  in  $V$
- **Double Negation:**  $\text{NOT}(\text{NOT}(v)) = v$  for all  $v$  in  $V$

These are not rules that logicians chose. They follow from the arithmetic. Check them yourself:  $0 \times (1-0) = 0$ .  $1 \times (1-1) = 0$ . Both zero. Non-Contradiction is just a multiplication fact.

*> Change one of the five things and different laws become forced. The same mechanism produces every known formal system. This book builds that mechanism in C.*

The threshold  $\theta$  measures *load*: how much work a reasoning step costs. Simple steps have low load. Deeply self-referential steps have high load. If the load exceeds  $\theta$ , the system cannot produce a coherent result. This is the mechanical explanation of Gödel's incompleteness theorems: some statements require load that grows without bound, exceeding any finite  $\theta$ . Not a philosophical mystery — a thermodynamic boundary.

The Liar Paradox (“This sentence is false”) demands a value  $v$  where  $v = \text{NOT}(v)$ . In  $\{0, 1\}$  that equation has no solution (it solves to  $v = 0.5$ , which is not in the carrier). Add 0.5 to the carrier and the paradox dissolves. One line of code in Chapter 4.

That is the framework. Now build it.

## Hello, Carrier in C

*Five things. Twenty lines. A complete logic.*

---

The classical carrier in C. Every construct used here is explained inline:

**carrier.c**

```
#include <stdio.h>
#include <math.h> /* fabs() */

/* The value carrier: an array of doubles and its length */
double V[] = {0.0, 1.0};
int V_N = 2; /* number of values in V */

/* The three operations */
double op_not(double v) { return 1.0 - v; }
double op_and(double a, double b) { return a * b; }
double op_or (double a, double b) { return a > b ? a : b; }

int main(void) {
    int i;
    for (i = 0; i < V_N; i++) {
        double v = V[i];
        printf("v=%.1f NOT=%.1f v AND NOT(v)=%.1f v OR NOT(v)=%.1f\n",
            v, op_not(v), op_and(v, op_not(v)), op_or(v, op_not(v)));
    }
    return 0;
}
```

```
$ gcc -Wall -std=c99 -lm -o carrier carrier.c && ./carrier
```

### OUTPUT

```
v=0.0 NOT=1.0 v AND NOT(v)=0.0 v OR NOT(v)=1.0 v=1.0 NOT=0.0 v AND NOT(v)=0.0 v OR
NOT(v)=1.0
```

The last two columns confirm Non-Contradiction (always 0) and Excluded Middle (always 1). Both laws are arithmetic consequences of the carrier. You verified them by running every value.



## C constructs used here

---

`double`: a 64-bit floating-point number. Used for all values because later chapters use values between 0 and 1.

`double v[] = {...}`: an array. `v[0]` is the first element, `v[1]` the second.

`int v_N = 2`: the length of the array. C arrays do not carry their own length; you track it separately.

`a > b ? a : b`: the ternary operator. Reads: if  $a > b$  then  $a$ , else  $b$ . This is `max(a, b)`.

`printf("%.1f", x)`: print a double with one decimal place.

### EXERCISE 2 · ★

Add a loop that also prints `NOT(NOT(v))` for each value and verifies it equals `v`. Use `fabs(not_not_v - v) < 1e-10` for the comparison and print whether it holds.

#### Answer:

```
for (i = 0; i < v_N; i++) {
    double v      = V[i];
    double not_v   = op_not(v);
    double not_not_v = op_not(not_v);
    int    holds   = fabs(not_not_v - v) < 1e-10;
    printf("v=%.1f NOT(NOT(v))=%.1f DN holds: %s\n",
          v, not_not_v, holds ? "yes" : "NO");
}
```

#### OUTPUT

```
v=0.0 NOT(NOT(v))=0.0 DN holds: yes v=1.0 NOT(NOT(v))=1.0 DN holds: yes
```

## Laws Are Tests

*A for-loop over all values is an exhaustive proof.*

---

To check whether a law holds, test every value. If it holds for all of them, it is forced. One counterexample disproves it. In C this is a for-loop that returns 0 (false) the moment a value fails:

```
carrier.c (updated)
```

```

#include <stdio.h>
#include <math.h>
#define TOL 1e-10    /* tolerance for floating-point comparisons */

double V[] = {0.0, 1.0};
int V_N = 2;

double op_not(double v)      { return 1.0 - v; }
double op_and(double a, double b) { return a * b; }
double op_or (double a, double b) { return a > b ? a : b; }

/* Returns 1 if LNC holds for all values, 0 otherwise */
int check_lnc(double *v, int n) {
    int i;
    for (i = 0; i < n; i++) {
        if (fabs(op_and(v[i], op_not(v[i]))) > TOL) {
            printf(" LNC fails at v=%.4f\n", v[i]);
            return 0;
        }
    }
    return 1;
}

int check_lem(double *v, int n) {
    int i;
    for (i = 0; i < n; i++) {
        double result = op_or(v[i], op_not(v[i]));
        if (fabs(result - 1.0) > TOL) {
            printf(" LEM fails at v=%.4f (OR gives %.4f)\n", v[i], result);
            return 0;
        }
    }
    return 1;
}

int check_dn(double *v, int n) {
    int i;
    for (i = 0; i < n; i++) {
        if (fabs(op_not(op_not(v[i])) - v[i]) > TOL) {
            printf(" DN fails at v=%.4f\n", v[i]);
            return 0;
        }
    }
    return 1;
}

int main(void) {
    printf("Classical carrier {0, 1}:\n");
    printf(" LNC: %s\n", check_lnc(V, V_N) ? "forced" : "FAILS");
    printf(" LEM: %s\n", check_lem(V, V_N) ? "forced" : "FAILS");
    printf(" DN: %s\n", check_dn (V, V_N) ? "forced" : "FAILS");
    return 0;
}

```

OUTPUT

Classical carrier {0, 1}: LNC: forced LEM: forced DN: forced

**C pattern:** functions that check a condition return `int`: `1` for true (success), `0` for false (failure). This is standard C convention. The function also prints diagnostic information before returning 0, so you know *where* it failed, not just *that* it failed.

### EXERCISE 3 · ★

Write `check_dn` using a different approach: collect all failing values into an array and print them all at once. Return the count of failures (0 means the law is forced).

#### Answer:

```
/* Returns number of DN failures (0 = forced) */
int check_dn_verbose(double *v, int n) {
    int i, failures = 0;
    for (i = 0; i < n; i++) {
        double result = op_not(op_not(v[i]));
        if (fabs(result - v[i]) > TOL) {
            printf("  DN fails at %.4f: NOT(NOT(%.4f)) = %.4f\n",
                v[i], v[i], result);
            failures++;
        }
    }
    return failures;
}

/* Usage: */
int dn_fails = check_dn_verbose(V, V_N);
printf("DN: %s (%d failures)\n", dn_fails == 0 ? "forced" : "FAILS", dn_fails);
```

## One More Value

Add 0.5 to the array. Watch two laws break.

Add one element to the array. Nothing else changes.

```
double V[] = {0.0, 0.5, 1.0};  /* added 0.5 */
int    V_N = 3;                /* updated count */
```

Recompile and run:

```
$ make && ./carrier
```

OUTPUT

```
Classical carrier {0, 0.5, 1}: LNC fails at v=0.5000 LNC: FAILS LEM fails at v=0.5000
(OR gives 0.5000) LEM: FAILS DN: forced
```

Two laws broke. The operations did not change. Only the value array changed. Let us verify why by hand:

```
/* NOT(0.5) = 1.0 - 0.5 = 0.5 -- it is its own negation */
/* AND(0.5, NOT(0.5)) = AND(0.5, 0.5) = 0.5 * 0.5 = 0.25 -- not zero */
/* OR(0.5, NOT(0.5)) = OR(0.5, 0.5) = max(0.5, 0.5) = 0.5 -- not 1 */
```

The value 0.5 is its own negation under  $\text{NOT}(v) = 1 - v$ . Multiplying it by itself gives 0.25, not 0. So LNC fails. The maximum of 0.5 and 0.5 is 0.5, not 1. So LEM fails. Double negation still works because  $1 - (1 - 0.5) = 0.5$ .

> The laws are arithmetic properties of the value set. Adding a value changes the arithmetic. The same operations produce different laws on different carriers. This is the central fact.

#### EXERCISE 4 · ★★

Try four values: `{0.0, 0.33, 0.67, 1.0}`. Predict which laws will break before running the code. Then run it. Were you right? At which values do LNC and LEM fail, and why?

**Answer:**

```
double V[] = {0.0, 0.33, 0.67, 1.0};  
int V_N = 4;
```

OUTPUT

```
LNC fails at v=0.3300 LNC: FAILS LEM fails at v=0.3300 (OR gives 0.6700) LEM:  
FAILS DN: forced
```

LNC fails at 0.33 because  $\text{NOT}(0.33) = 0.67$ , and  $\text{AND}(0.33, 0.67) = 0.33 * 0.67 = 0.221$ , not zero. LEM fails at 0.33 because  $\text{OR}(0.33, 0.67) = 0.67$ , not 1.0. The failures continue at 0.67 for the same reason. DN holds because  $1 - (1 - v) = v$  algebraically for any  $v$ .

## The Liar Paradox

*Two thousand years. One for-loop.*

---

The Liar Paradox demands a value  $v$  where  $v = \text{NOT}(v)$ . This is called a *fixed point* of NOT. Finding whether one exists is a linear search:

```
/* Returns the fixed point if found, -999.0 as sentinel if not */
double liar_solution(double *v, int n) {
    int i;
    for (i = 0; i < n; i++) {
        if (fabs(v[i] - op_not(v[i])) < TOL)
            return v[i];
    }
    return -999.0; /* sentinel: no solution */
}

void report_liar(double *v, int n) {
    double sol = liar_solution(v, n);
    if (sol > -998.0)
        printf(" Liar: resolved at v=%.4f\n", sol);
    else
        printf(" Liar: no fixed point -- paradox in this carrier\n");
}

int main(void) {
    double V2[] = {0.0, 1.0};
    double V3[] = {0.0, 0.5, 1.0};

    printf("Carrier {0, 1}:\n");
    report_liar(V2, 2);

    printf("Carrier {0, 0.5, 1}:\n");
    report_liar(V3, 3);

    return 0;
}
```

### OUTPUT

```
Carrier {0, 1}: Liar: no fixed point -- paradox in this carrier
Carrier {0, 0.5, 1}:
Liar: resolved at v=0.5000
```

The equation  $v = 1 - v$  solves to  $v = 0.5$ . The carrier  $\{0, 1\}$  does not contain 0.5, so there is no solution. The carrier  $\{0, 0.5, 1\}$  does contain 0.5, so the paradox resolves. The Liar is a statement about which value space you are working in, not about truth itself.

> A paradox is not a flaw in logic. It is the value space not containing the solution the equation requires. Extend the carrier and it dissolves.

## Continuous search

---

For large or continuous carriers, generate the values in a loop:

```
/* Generate 1001 evenly spaced values in [0, 1] */
#define N_CONT 1001
double cont[N_CONT];
int i;
for (i = 0; i < N_CONT; i++)
    cont[i] = i / (N_CONT - 1.0);

report_liar(cont, N_CONT);
```

OUTPUT

Liar: resolved at v=0.5000



### EXERCISE 5 · ★★

Write a function `all_liar_solutions` that fills a provided array with all fixed points and returns the count. Test it on the four-valued carrier `{0.0, 0.33, 0.67, 1.0}`.

#### Answer:

```
/* Fills `out` with all fixed points, returns count */
int all_liar_solutions(double *v, int n, double *out, int out_max) {
    int i, count = 0;
    for (i = 0; i < n && count < out_max; i++) {
        if (fabs(v[i] - op_not(v[i])) < TOL)
            out[count++] = v[i];
    }
    return count;
}

double V4[] = {0.0, 0.33, 0.67, 1.0};
double sols[16];
int n_sols = all_liar_solutions(V4, 4, sols, 16);
printf("%d Liar solutions\n", n_sols);
```

#### OUTPUT

```
0 Liar solutions
```

No solutions in `{0, 0.33, 0.67, 1}`. The equation `v = 1-v` requires `v = 0.5`, which is not in this carrier. The Liar is still a paradox here. Only carriers containing 0.5 resolve it under this NOT operation.

## The Price of Thinking

*Every operation costs something. The threshold is the budget.*

---

Maintaining a distinction between true and false requires thermodynamic work. At room temperature, erasing one bit of information costs at minimum  $2.87 \times 10^{-21}$  joules (the Landauer limit). We track this abstractly as *load*: a non-negative number that accumulates as patterns propagate.

The *coherence threshold*  $\theta$  is the maximum load the system can tolerate. A pattern with load above  $\theta$  cannot settle into a stable state. It is incoherent.

Load combination rules:

- **NOT(P)**: load unchanged — negation is free
- **AND(P, Q)**: loads add ( $L_P + L_Q$ ) — conjunction is expensive
- **OR(P, Q)**: load takes minimum ( $\min(L_P, L_Q)$ ) — takes the cheaper path

```

#define THETA 10.0  /* coherence threshold */

/* Check whether a load is coherent */
int is_coherent(double load) {
    return load <= THETA;
}

/* Load combination for AND */
double load_and(double L1, double L2) { return L1 + L2; }

/* Load combination for OR */
double load_or(double L1, double L2) { return L1 < L2 ? L1 : L2; }

int main(void) {
    double loads[] = {2.0, 3.0, 2.5, 4.0};
    double acc      = loads[0];
    int      i;

    printf("AND chain (theta=%.1f):\n", THETA);
    printf("  Start:  load=%.1f coherent=%s\n",
        acc, is_coherent(acc) ? "yes" : "NO");

    for (i = 1; i < 4; i++) {
        acc = load_and(acc, loads[i]);
        printf("  Step %d: load=%.1f coherent=%s\n",
            i, acc, is_coherent(acc) ? "yes" : "NO");
        if (!is_coherent(acc)) {
            printf("  Stopped: threshold exceeded.\n");
            break;
        }
    }
    return 0;
}

```

#### OUTPUT

```

AND chain (theta=10.0): Start: load=2.0 coherent=yes Step 1: load=5.0 coherent=yes
Step 2: load=7.5 coherent=yes Step 3: load=11.5 coherent=NO Stopped: threshold
exceeded.

```

## EXERCISE 6 · ★★

Run the same chain using `load_or` instead. Does it ever exceed the threshold? Explain why in terms of the min operation.

**Answer:**

```
acc = loads[0];
printf("OR chain (theta=%.1f):\n", THETA);
for (i = 1; i < 4; i++) {
    acc = load_or(acc, loads[i]);
    printf(" Step %d: load=%.1f\n", i, acc);
}
```

OUTPUT

```
OR chain (theta=10.0): Step 1: load=2.0 Step 2: load=2.0 Step 3: load=2.0
```

OR always takes the minimum load, so a chain of ORs never increases beyond the smallest load in the chain. The path of least resistance is always taken. This is why disjunction is structurally cheaper than conjunction.

## Loaded Patterns

$P = (value, load)$ . The propagation operator in  $C$ .

---

A *pattern* is a value paired with a load. In  $C$ , this is a `struct`:

```

/* A loaded pattern: value and load */
typedef struct {
    double value;
    double load;
} Pattern;

/* Constructors */
Pattern pat(double v, double l) { Pattern p; p.value = v; p.load = l; return p; }
Pattern pat_v(double v)          { return pat(v, 0.0); } /* zero load */

/* Operations on patterns */
Pattern pat_not(Pattern p) {
    return pat(1.0 - p.value, p.load); /* load unchanged */
}

Pattern pat_and(Pattern p, Pattern q) {
    return pat(p.value * q.value, p.load + q.load); /* loads add */
}

Pattern pat_or(Pattern p, Pattern q) {
    double min_load = p.load < q.load ? p.load : q.load;
    double max_val  = p.value > q.value ? p.value : q.value;
    return pat(max_val, min_load); /* cheapest path */
}

/* The propagation operator: P / G -> Q */
typedef Pattern (*Gradient1)(Pattern);
typedef Pattern (*Gradient2)(Pattern, Pattern);

Pattern propagate1(Pattern p, Gradient1 G) { return G(p); }
Pattern propagate2(Pattern p, Pattern q, Gradient2 G) { return G(p, q); }

int main(void) {
    Pattern P = pat(1.0, 3.0);
    Pattern Q = pat(1.0, 5.0);
    Pattern R;

    printf("P = (%.1f, %.1f)\n", P.value, P.load);
    printf("Q = (%.1f, %.1f)\n", Q.value, Q.load);

    R = propagate2(P, Q, pat_and);
    printf("P / AND -> (%.1f, %.1f)\n", R.value, R.load);

    R = propagate2(P, Q, pat_or);
    printf("P / OR  -> (%.1f, %.1f)\n", R.value, R.load);

    R = propagate1(P, pat_not);
    printf("P / NOT -> (%.1f, %.1f)\n", R.value, R.load);

    return 0;
}

```

OUTPUT

```
P = (1.0, 3.0) Q = (1.0, 5.0) P / AND -> (1.0, 8.0) P / OR -> (1.0, 3.0) P / NOT ->
(0.0, 3.0)
```

**C pattern: function pointers.** `typedef Pattern (*Gradient1)(Pattern)` declares a type for functions that take one Pattern and return a Pattern. You can then pass any such function as an argument to `propagate1`. This is how C implements first-class functions.

#### EXERCISE 7 · ★★

Add an implication operation `pat_implies(Pattern p, Pattern q)`: if `p.value == 1.0`, return `q`; otherwise return `pat(1.0, 0.0)` (vacuously true, zero cost). Print the result for all four value combinations of P and Q in `{0, 1}`.

#### Answer:

```
Pattern pat_implies(Pattern p, Pattern q) {
    if (p.value > 0.5)          /* p is designated */
        return q;
    return pat(1.0, 0.0);      /* vacuously true, free */
}

double vals[] = {0.0, 1.0};
int a, b;
for (a = 0; a < 2; a++)
    for (b = 0; b < 2; b++) {
        Pattern p = pat(vals[a], 1.0);
        Pattern q = pat(vals[b], 1.0);
        Pattern r = pat_implies(p, q);
        printf("%.0f -> %.0f = %.0f (load=%.1f)\n",
               p.value, q.value, r.value, r.load);
    }
```

#### OUTPUT

```
0 -> 0 = 1 (load=0.0) 0 -> 1 = 1 (load=0.0) 1 -> 0 = 0 (load=1.0) 1 -> 1 = 1
(load=1.0)
```

Matches the classical truth table. Vacuous truths ( $P=0$ ) cost nothing. Forced implications ( $P=1$ ) carry the load of Q. The only failure: P is true and Q is false.

## The Carrier Struct

*Five fields. One object. A complete formal system.*

---

Collect the five carrier components into a struct with function pointers for the operations:



```

#include <stdio.h>
#include <math.h>
#include <string.h>
#define TOL 1e-10
#define MAX_V 256
#define NAME_LEN 64

typedef struct {
    char name[NAME_LEN];
    double v[MAX_V];
    int n;
    double (*not_fn)(double);
    double (*and_fn)(double, double);
    double (*or_fn)(double, double);
    double theta;
} Carrier;

/* Build a carrier from its components */
Carrier make_carrier(
    const char *name,
    double *vals, int n,
    double (*not_fn)(double),
    double (*and_fn)(double, double),
    double (*or_fn)(double, double),
    double theta)
{
    Carrier c;
    strncpy(c.name, name, NAME_LEN - 1);
    c.name[NAME_LEN-1] = '\0';
    memcpy(c.v, vals, n * sizeof(double));
    c.n = n; c.not_fn = not_fn; c.and_fn = and_fn;
    c.or_fn = or_fn; c.theta = theta;
    return c;
}

/* Law checks on a Carrier */
int carrier_lnc(Carrier *c) {
    int i;
    for (i = 0; i < c->n; i++)
        if (fabs(c->and_fn(c->v[i], c->not_fn(c->v[i]))) > TOL) return 0;
    return 1;
}

int carrier_lem(Carrier *c) {
    int i; double top = c->v[0];
    for (i = 1; i < c->n; i++) if (c->v[i] > top) top = c->v[i];
    for (i = 0; i < c->n; i++)
        if (fabs(c->or_fn(c->v[i], c->not_fn(c->v[i])) - top) > TOL) return 0;
    return 1;
}

int carrier_dn(Carrier *c) {
    int i;
    for (i = 0; i < c->n; i++)
        if (fabs(c->not_fn(c->not_fn(c->v[i])) - c->v[i]) > TOL) return 0;
    return 1;
}

```

```

}

double carrier_liar(Carrier *c) {
    int i;
    for (i = 0; i < c->n; i++)
        if (fabs(c->v[i] - c->not_fn(c->v[i])) < TOL) return c->v[i];
    return -999.0;
}

void carrier_report(Carrier *c) {
    double liar;
    printf("\n%s\n", c->name);
    printf("  n=%d  theta=%.1f\n", c->n, c->theta);
    printf("  LNC: %s\n", carrier_lnc(c) ? "forced" : "FAILS");
    printf("  LEM: %s\n", carrier_lem(c) ? "forced" : "FAILS");
    printf("  DN: %s\n", carrier_dn(c) ? "forced" : "FAILS");
    liar = carrier_liar(c);
    if (liar > -998.0)
        printf("  Liar: resolved at %.4f\n", liar);
    else
        printf("  Liar: paradox (no fixed point)\n");
}

/* Operations */
double std_not(double v)          { return 1.0 - v; }
double std_and(double a, double b) { return a * b; }
double std_or (double a, double b) { return a > b ? a : b; }

int main(void) {
    double v2[] = {0.0, 1.0};
    double v3[] = {0.0, 0.5, 1.0};

    Carrier classical = make_carrier("Classical",
        v2, 2, std_not, std_and, std_or, 1.0);
    Carrier para      = make_carrier("Paraconsistent",
        v3, 3, std_not, std_and, std_or, 1.0);

    carrier_report(&classical);
    carrier_report(&para);
    return 0;
}

```

#### OUTPUT

```

Classical n=2 theta=1.0 LNC: forced LEM: forced DN: forced Liar: paradox (no fixed
point) Paraconsistent n=3 theta=1.0 LNC: FAILS LEM: FAILS DN: forced Liar: resolved at
0.5000

```

**C pattern: pointers to structs.** `carrier_report(Carrier *c)` takes a pointer. Access fields with

`c->name` (arrow notation). Passing a pointer avoids copying the entire struct on every call.

Always pass large structs by pointer in C.

## EXERCISE 8 · ★★

Add a `carrier_closure` function that checks whether NOT maps every value back into V (within TOL). It should print which values (if any) map outside V. Test it on the standard carriers and on one where you deliberately break closure by returning 2.0 for NOT(1.0).

### Answer:

```
int carrier_closure(Carrier *c) {
    int i, j, found;
    for (i = 0; i < c->n; i++) {
        double result = c->not_fn(c->v[i]);
        found = 0;
        for (j = 0; j < c->n; j++)
            if (fabs(c->v[j] - result) < TOL) { found = 1; break; }
        if (!found) {
            printf(" Closure fails: NOT(%.4f)=%.4f not in V\n",
                   c->v[i], result);
            return 0;
        }
    }
    return 1;
}
```

## Comparing Two Carriers

*A diff function. What changes. What survives.*

The central claim: change one parameter, some laws change, others do not. Verify it mechanically:

```
void carrier_diff(Carrier *c1, Carrier *c2) {
    int lnc1 = carrier_lnc(c1), lnc2 = carrier_lnc(c2);
    int lem1 = carrier_lem(c1), lem2 = carrier_lem(c2);
    int dn1 = carrier_dn(c1), dn2 = carrier_dn(c2);
    int li1 = carrier_liar(c1) > -998.0;
    int li2 = carrier_liar(c2) > -998.0;

    printf("\nDiff: %s vs %s\n", c1->name, c2->name);
    printf("  %-20s %s: %C    %s: %C%s\n", "LNC",
        c1->name, lnc1?'Y':'N', c2->name, lnc2?'Y':'N',
        lnc1!=lnc2 ? "  <-- CHANGED" : "");
    printf("  %-20s %s: %C    %s: %C%s\n", "LEM",
        c1->name, lem1?'Y':'N', c2->name, lem2?'Y':'N',
        lem1!=lem2 ? "  <-- CHANGED" : "");
    printf("  %-20s %s: %C    %s: %C%s\n", "DN",
        c1->name, dn1?'Y':'N', c2->name, dn2?'Y':'N',
        dn1!=dn2 ? "  <-- CHANGED" : "");
    printf("  %-20s %s: %C    %s: %C%s\n", "Liar resolved",
        c1->name, li1?'Y':'N', c2->name, li2?'Y':'N',
        li1!=li2 ? "  <-- CHANGED" : "");
}

int main(void) {
    double v2[] = {0.0, 1.0};
    double v3[] = {0.0, 0.5, 1.0};
    Carrier classical = make_carrier("Classical", v2, 2, std_not, std_and, std_or, 1.0);
    Carrier para = make_carrier("Paraconsistent", v3, 3, std_not, std_and, std_or, 1.0);
    carrier_diff(&classical, &para);
    return 0;
}
```

### OUTPUT

```
Diff: Classical vs Paraconsistent LNC Classical: Y Paraconsistent: N <-- CHANGED LEM
Classical: Y Paraconsistent: N <-- CHANGED DN Classical: Y Paraconsistent: Y Liar
resolved Classical: N Paraconsistent: Y <-- CHANGED
```

### EXERCISE 9 · ★★

Build a third carrier with the same values as classical `{0, 1}` but with `AND = min` instead of multiply. Diff it against classical. Which laws change when you change AND but keep V the same?

**Answer:**

```
double min_and(double a, double b) { return a < b ? a : b; }
double v2[] = {0.0, 1.0};
Carrier c_min = make_carrier("Classical-minAND",
    v2, 2, std_not, min_and, std_or, 1.0);
carrier_diff(&classical, &c_min);
```

#### OUTPUT

```
Diff: Classical vs Classical-minAND LNC Classical: Y Classical-minAND: Y LEM
Classical: Y Classical-minAND: Y DN Classical: Y Classical-minAND: Y Liar
resolved Classical: N Classical-minAND: N
```

No change. On `{0, 1}`, `min(a,b)` and `a*b` are identical: `min(0,1)=0=0*1`, `min(1,1)=1=1*1`. Operations diverge only when values between 0 and 1 exist in the carrier. Same operation change, different carrier, different result.

## The Paradox Classifier

*Four modes. Integer flags. No magic.*

---

There are exactly four ways a formal system can break. C uses integer flags and a struct to classify them:

```

#define MODE1_LIAR    1
#define MODE2_GOEDEL  2
#define MODE3_RUSSELL 4
#define MODE4_YABLO   8

typedef struct {
    int    flags;           /* bitfield of active paradox modes */
    double liar_solution;   /* -999 if not resolved */
    int    goedel_depth;    /* depth at which load exceeds theta */
} ParadoxReport;

ParadoxReport classify(Carrier *c, double theta, int max_depth) {
    ParadoxReport r;
    double load;
    int    depth;

    r.flags = 0;

    /* Mode 1: Liar -- does NOT have a fixed point? */
    r.liar_solution = carrier_liar(c);
    if (r.liar_solution < -998.0)
        r.flags |= MODE1_LIAR;

    /* Mode 2: Goedel/Turing -- self-referential load diverges */
    load = 1.0;
    r.godel_depth = -1;
    for (depth = 1; depth <= max_depth; depth++) {
        load *= 2.0;
        if (load > theta) {
            r.flags |= MODE2_GOEDEL;
            r.godel_depth = depth;
            break;
        }
    }

    /* Mode 4: Yablo -- no seed state (always present) */
    r.flags |= MODE4_YABLO;

    return r;
}

void print_report(Carrier *c, ParadoxReport *r) {
    printf("\nParadox classification: %s\n", c->name);
    printf("  Mode 1 (Liar):    %s",
        (r->flags & MODE1_LIAR) ? "PARADOX" : "resolved");
    if (!(r->flags & MODE1_LIAR))
        printf(" at %.4f", r->liar_solution);
    printf("\n");
    printf("  Mode 2 (Goedel):  %s",
        (r->flags & MODE2_GOEDEL) ? "PARADOX" : "bounded");
    if (r->flags & MODE2_GOEDEL)
        printf(" (diverges at depth %d)", r->goedel_depth);
    printf("\n");
    printf("  Mode 4 (Yablo):   PARADOX (no seed -- universal)\n");
}

```

```

int main(void) {
    double v2[] = {0.0, 1.0};
    double v3[] = {0.0, 0.5, 1.0};
    Carrier    classical = make_carrier("Classical",    v2, 2, std_not, std_and,
std_or, 1.0);
    Carrier    para      = make_carrier("Paraconsistent", v3, 3, std_not, std_and,
std_or, 1.0);
    ParadoxReport r1      = classify(&classical, 1000.0, 20);
    ParadoxReport r2      = classify(&para,      1000.0, 20);
    print_report(&classical, &r1);
    print_report(&para,      &r2);
    return 0;
}

```

#### OUTPUT

Paradox classification: Classical Mode 1 (Liar): PARADOX Mode 2 (Goedel): PARADOX (diverges at depth 10) Mode 4 (Yablo): PARADOX (no seed -- universal) Paradox classification: Paraconsistent Mode 1 (Liar): resolved at 0.5000 Mode 2 (Goedel): PARADOX (diverges at depth 10) Mode 4 (Yablo): PARADOX (no seed -- universal)

> Mode 1 resolves by extending the carrier. Modes 2 and 4 survive all carrier extensions. They require changes to what the system can express, not just what values it contains.

#### EXERCISE 10 · ★★★

Add Mode 3 (Russell) as a check: a carrier exhibits a Russell-type collapse if it contains a value equal to its own length (a self-referential size). Add this check to `classify` and test it on a carrier you construct to trigger it.

#### Answer:

```

/* Mode 3: value equals carrier size (self-referential size proxy) */
int i;
for (i = 0; i < c->n; i++) {
    if (fabs(c->v[i] - c->n) < TOL) {
        r.flags |= MODE3_RUSSELL;
        break;
    }
}

/* Trigger it: carrier {0, 1, 3} has len=3 and contains value 3 */
double v_russell[] = {0.0, 1.0, 3.0};
Carrier russell = make_carrier("Russell-trigger",
    v_russell, 3, std_not, std_and, std_or, 1.0);

```



## Build Your Own

*The blank template. Five exercises. Start here.*

---

my\_carrier.c

```
#include <stdio.h>
#include <math.h>
#include <string.h>
#define TOL 1e-10
#define MAX_V 256

/* === Paste the Carrier struct and all functions from Chapter 8 here === */

/* === Define your operations === */
double my_not(double v) {
    /* TODO: return the negation of v */
    return 0.0;
}

double my_and(double a, double b) {
    /* TODO: return the conjunction */
    return 0.0;
}

double my_or(double a, double b) {
    /* TODO: return the disjunction */
    return 0.0;
}

int main(void) {
    double my_vals[] = { /* TODO: your values */ 0.0, 1.0 };
    int my_n = 2; /* TODO: count of values */

    Carrier c = make_carrier("My Carrier",
        my_vals, my_n, my_not, my_and, my_or, 1.0);
    carrier_report(&c);
    return 0;
}
```

### Five build exercises

---

**L1 ★** Build the classical carrier using the template. Verify all four laws pass and the Liar is a paradox. Then add 0.5 to the values array. Recompile. Verify LNC and LEM fail and the Liar resolves.

**L2 ★★** Build a four-valued logic for a weather forecast: 0.0 = no rain, 0.33 = possible, 0.67 = likely, 1.0 = certain. Define NOT, AND, OR to reflect forecast combination rules (AND = most cautious, OR = most optimistic). Check all laws. Does the Liar resolve?

**L3 ★★** Write a function `search_lnc_and` that, for a fixed carrier and NOT, tries every possible AND operation from a family and reports which ones force LNC. Use AND = multiply, AND = min, AND = max, and at least one you invent yourself.

**L4 ★★★** Build a continuous carrier with 10001 values from 0 to 1. Check all three laws with a tolerance of 1e-8. For LNC, count and print the exact number of failures and the first three failing values. Then increase the carrier size to 100001 and time the difference using the `time` command: `time ./carrier`.

**L5 ★★★** Implement a command-line interface for your carrier tool. If `argv[1]` is `"classical"`, report the classical carrier. If it is `"para"`, report the paraconsistent carrier. If it is `"diff"`, diff them. Handle unknown arguments with a usage message. This makes your carrier tool a proper Linux command.

# The Complete Framework

*All code in one file. Copy, compile, build on.*



`carrier_framework.c`

```

/*
 * carrier_framework.c
 * Propagation Logic – Complete Carrier Set Framework in C
 * James Alexander Pugmire · 2026
 * Build: gcc -Wall -Wextra -std=c99 -lm -o carrier carrier_framework.c
 * Run: ./carrier
 */

#include <stdio.h>
#include <math.h>
#include <string.h>

#define TOL 1e-10
#define MAX_V 256
#define NAME_LEN 64

/* — Pattern — */
typedef struct { double value; double load; } Pattern;
Pattern pat(double v, double l) { Pattern p; p.value=v; p.load=l; return p; }

Pattern pat_not(Pattern p) { return pat(1.0-p.value, p.load); }
Pattern pat_and(Pattern p, Pattern q) { return pat(p.value*q.value, p.load+q.load); }
Pattern pat_or (Pattern p, Pattern q) {
    return pat(p.value>q.value?p.value:q.value, p.load/* — Carrier
    */
};

typedef struct {
    char name[NAME_LEN];
    double v[MAX_V];
    int n;
    double (*not_fn)(double);
    double (*and_fn)(double, double);
    double (*or_fn)(double, double);
    double theta;
} Carrier;

Carrier make_carrier(const char *name, double *vals, int n,
    double(*not_fn)(double), double(*and_fn)(double,double),
    double(*or_fn)(double,double), double theta)
{
    Carrier c;
    strncpy(c.name, name, NAME_LEN-1); c.name[NAME_LEN-1]='\0';
    memcpy(c.v, vals, n*sizeof(double));
    c.n=n; c.not_fn=not_fn; c.and_fn=and_fn; c.or_fn=or_fn; c.theta=theta;
    return c;
}

/* — Law checks — */
int carrier_lnc(Carrier*c) {
    int i; for(i=0;i<c->n;i++) if(fabs(c->and_fn(c->v[i],c->not_fn(c->v[i]))>c->not_fn(c->v[i]))>TOL)return 0; return 1; }
int carrier_lem(Carrier*c) {
    int i; double top=c->v[0];
    for(i=1;i<c->n;i++) if(c->v[i]>top)top=c->v[i];
    for(i=0;i<c->n;i++) if(fabs(c->or_fn(c->v[i],c->not_fn(c->v[i]))-top)>TOL)return 0;
    return 1; }
int carrier_dn(Carrier*c) {

```

```

    int i; for(i=0;i<c->n;i++) if(fabs(c->not_fn(c->not_fn(c->v[i]))-c->v[i])>TOL)return 0; return 1; }
double carrier_liar(Carrier*c) {
    int i; for(i=0;i<c->n;i++) if(fabs(c->v[i]-c->not_fn(c->v[i]))<TOL)return c->v[i]; return -999.0; }

void carrier_report(Carrier*c) {
    double liar = carrier_liar(c);
    printf("\n%s (n=%d theta=%.1f)\n", c->name, c->n, c->theta);
    printf(" LNC: %s\n", carrier_lnc(c)?"forced":"FAILS");
    printf(" LEM: %s\n", carrier_lem(c)?"forced":"FAILS");
    printf(" DN: %s\n", carrier_dn(c)?"forced":"FAILS");
    printf(" Liar: %s\n", liar>-998.0?"resolved":"paradox");
}

void carrier_diff(Carrier*c1, Carrier*c2) {
    int r1[4] = {carrier_lnc(c1),carrier_lem(c1),carrier_dn(c1),carrier_liar(c1)>-998.0};
    ;
    int r2[4] = {carrier_lnc(c2),carrier_lem(c2),carrier_dn(c2),carrier_liar(c2)>-998.0};
    ;
    const char *laws[4] = {"LNC","LEM","DN","Liar"};
    int i;
    printf("\nDiff: %s vs %s\n", c1->name, c2->name);
    for(i=0;i<4;i++)
        printf(" %-8s %s:%c %s:%c%s\n", laws[i],
            c1->name, r1[i]?'Y':'N', c2->name, r2[i]?'Y':'N',
            r1[i]!=r2[i]?" <-- ":"");
}

/* — Standard operations ————— */
double std_not(double v) { return 1.0-v; }
double std_and(double a, double b) { return a*b; }
double std_or (double a, double b) { return a>b?a:b; }

/* — Main ————— */
int main(void) {
    double v2[] = {0.0,1.0};
    double v3[] = {0.0,0.5,1.0};

    Carrier classical = make_carrier("Classical", v2,2,std_not,std_and,std_or,1.0);
    Carrier para = make_carrier("Paraconsistent", v3,3,std_not,std_and,std_or,1.0);

    carrier_report(&classical);
    carrier_report(&para);
    carrier_diff (&classical, &para);
    return 0;
}

```

## Makefile Reference

*Build, clean, and extend your project.*

---

### Makefile

```
CC      = gcc
CFLAGS  = -Wall -Wextra -std=c99 -lm

# Default target: build the main carrier tool
carrier: carrier_framework.c
        $(CC) $(CFLAGS) -o carrier carrier_framework.c

# Build your custom carrier
my_carrier: my_carrier.c carrier_framework.c
        $(CC) $(CFLAGS) -o my_carrier my_carrier.c

# Build with debug symbols (for gdb)
debug: carrier_framework.c
        $(CC) $(CFLAGS) -g -o carrier_debug carrier_framework.c

# Run all law checks (add your own as you build them)
test: carrier
        ./carrier

clean:
        rm -f carrier my_carrier carrier_debug

.PHONY: clean test debug
```

### Common commands:

```
make # build carrier
make test # build and run
make clean # remove binaries
make debug # build with debug symbols
gcc -Wall -std=c99 -lm -o carrier carrier_framework.c # direct compile
```

## The Mathematics: Reference

*Formal definitions. For when you want to go deeper.*

---

### The three parameters

---

Every formal system is completely specified by: **V** (value carrier), **Γ** (gradient family, the set of available operations), and **θ** (coherence threshold). Change one and different laws are forced. The mechanism does not change.

### Forced laws

---

A statement is *forced* in a carrier if it holds for every value in V. Testing is exhaustive: one counterexample disproves, no counterexample across all values proves.

- **Law of Non-Contradiction (LNC):**  $v \text{ AND } \text{NOT}(v) = \emptyset$  for all  $v$  in V
- **Law of Excluded Middle (LEM):**  $v \text{ OR } \text{NOT}(v) = \max(V)$  for all  $v$  in V
- **Double Negation (DN):**  $\text{NOT}(\text{NOT}(v)) = v$  for all  $v$  in V

### Load combination rules

---

Each pattern carries a load:  $P = (\text{value}, \text{load})$ . Operations combine loads as follows:

- **NOT(P):** load unchanged
- **AND(P, Q):** load =  $L_P + L_Q$  (conjunction is expensive)
- **OR(P, Q):** load =  $\min(L_P, L_Q)$  (takes the cheaper path)

### The four paradox modes

---

Every paradox in formal logic is one of four gradient-overload modes:

- **Mode 1 — Liar:** NOT has no fixed point in V ( $v = \text{NOT}(v)$  has no solution). Resolved by extending V to contain 0.5.
- **Mode 2 — Gödel/Turing:** self-referential load doubles at each depth.  $L(d) = 2 \supset d$ . Exceeds any finite  $\theta$ . Not resolved by carrier extension.

- **Mode 3 — Russell:** the gradient family demands a context larger than the one it defines. Level collapse. Resolved by type stratification (higher axiom debt).
- **Mode 4 — Yablo:** no seed state. Construction without a base case. Not resolved by carrier extension. No self-reference required.

---

### **DRAS: every quantity is a loaded history**

---

The De-Reification Axiom Standard observes that every measurable quantity is a loaded history  $q(E)$  evaluated at energy scale  $E$ , not a context-free constant. The fine structure constant is approximately  $1/137$  at low energies and  $1/129$  near the  $Z$  boson mass. There are no constants. There are only measurements at specific scales.

In formal terms: the carrier  $V$  should be written  $V(E)$  — a value space that may depend on the scale at which it is evaluated. Most formal systems assume scale independence. Physics abandoned that assumption in the 1970s.

---

### **The propagation operator**

---

The complete formal statement: a loaded pattern  $P = (v_P, L_P)$  propagates through a gradient field  $G$  in context  $C = (\Gamma, \theta)$  to produce output pattern  $Q = P / G$  where:

- **value(Q)** =  $G$  applied to  $\text{value}(P)$  per the operation's value rule
- **load(Q)** =  $G$  applied to  $\text{load}(P)$  per the operation's load rule
- **Q is coherent** iff  $\text{load}(Q) \leq \theta$

All of logic, calculus, and probability theory are instances of this operator on different carriers. This is not analogy. It is the same arithmetic in different value spaces.